Plan d'action – Projet POO (Player, Warrior, Mage, Game)

Analyse des consignes (README, guide d'architecture, convention de style, diagramme) et du code actuel, avec un plan priorisé pour la partie Player/Game, sachant que le binôme s'occupe d'Enemy et de Map.

État des lieux de ton code

Bloquant

- `Warrior.h` déclare `GetHP()` et `Maxhp()` en `override`, mais `Warrior.cpp` ne définit que `Move`. Le projet ne se lie donc pas, parce que `main` construit un `Warrior`.
- Les opérateurs de `Vector2` (`+`, `=`, `==`) sont déclarés mais jamais définis. Il n'y a pas non plus de constructeur par défaut. Le binôme va se heurter à ça avec `Vector2 playerSpawn;` dans `Map`.

Écarts avec les consignes (donc des points en moins)

- `Entity` est trop vide. Le guide attend `onUpdate`, `onCollision(Entity*)`, `applyDamage(int)` et `isAlive`.
- `hp` et `position` sont dans `Player`. Le binôme va devoir les dupliquer dans `Enemy`, alors qu'ils devraient être dans `Entity`.
- `Move` est redéfini et vide dans `Warrior` et `Mage`. Ce que le guide fait varier entre les deux, c'est `onUpdate` et `onCollision`, pas le déplacement. La notation sanctionne explicitement le « virtual inutile ».
- `Maxhp()` renvoie 100 en dur. Or le Mage doit avoir **1 PV** (il en a 80 dans le `main`) et le Warrior plusieurs.
- `Player` est instancié dans `main`. Il doit devenir abstrait.
- `Game` est vide, et `main` doit se limiter à `Game::GetInstance()->Run()`.
- Style : tabulations au lieu de 4 espaces, `GetHP` / `Maxhp` au lieu du camelCase (`getHp`, `getMaxHp`), `protected` placé avant `public` dans `Player.h`, `#include <iostream>` inutiles.

Plan d'action

Étape 1 : débloquer le binôme (à faire en premier)

Son `Enemy` dépend de `Entity`, donc c'est ce qui compte le plus.

1. Corriger `Vector2` : constructeur par défaut `Vector2(int x = 0, int y = 0)` et opérateurs définis (inline dans le `.h`).
2. Remettre `Entity` d'équerre avec le binôme, en se mettant d'accord sur l'interface :
   - données dans `Entity` : `position`, `hp`, `maxHp`, plus un booléen « peut nager » (le guide le propose pour ZombieSwimmer) ;
   - fonctions : `applyDamage(int)`, `isAlive()`, `move(dir)` **non virtuelle** (le même code sert au joueur et aux ennemis), `onUpdate()` et `onCollision(Entity*)` virtuelles pures.
3. Faire un commit sur la branche `player` (créée depuis `dev`), puis merge dans `dev` pour que l'autre parte de la même base.

Étape 2 : cœur du travail (Entity, Player, Warrior, Mage)

Cela vaut 30 points avec Entity-Player-Enemy, et environ la moitié des 30 points de Warrior-Mage-Zombie-HeadlessZombie.

1. `Move` : calculer la nouvelle position, vérifier les bordures, la case marchable, puis les collisions. L'écrire une seule fois dans `Entity`, en découpant en petites fonctions (limite de 25 lignes). Il a besoin de la `Map` du binôme, donc fixer ensemble une signature du genre `isWalkable(pos, canSwim)`.
2. `Player` : ajouter `readPlayerInputs()` (ZQSD + action) et `getDirection(action)`.
3. `Warrior` : plusieurs PV, et `onCollision` tue l'ennemi.
4. `Mage` : 1 PV, et `onUpdate` tue les ennemis adjacents quand on appuie sur espace, puis passe le tour.
5. Supprimer les `Move` vides de `Warrior` et `Mage`, et faire passer le `Maxhp` par le constructeur.

Étape 3 : `Game` et `main`, avec le binôme (après le merge `player` + `enemies`)

Le git flow impose une branche `game` à ce moment-là, et Map-Game vaut 20 points.

1. `Game` en Singleton (`GetInstance`, destructeur qui fait les `delete`).
2. `Run()` : boucle tour par tour joueur puis ennemis, qui supprime les entités mortes.
3. `main` ne contient plus que les 3 lignes du guide.

Étape 4 : `WarriorMage` (10 points, rentable)

C'est court si Warrior et Mage sont propres.

- Héritage virtuel : `Warrior` et `Mage` héritent de `Player` avec `virtual`, et `WarriorMage` appelle directement le constructeur de `Player`.

- `WarriorMage` redéfinit `onCollision` en appelant `Warrior::onCollision`, et `onUpdate` en appelant celui du Mage. Il ne copie aucun code, comme le guide l'exige.

Étape 5 : nettoyage avant rendu (évite les malus)

- Aucune ligne de plus de 90 caractères, aucune fonction de plus de 25 lignes (−10 chacune).
- Convention de style appliquée à tout le code, sans accents dans les noms.
- Git flow respecté : `dev`, puis `player` / `enemies` / `game`, puis `main`. Ajouter aussi un `.gitignore` pour `.vs`, `x64` et `*.user`, car le `.vcxproj.user` n'a rien à faire dans le dépôt.
- Aucune classe inutile : chaque classe doit pouvoir être justifiée à l'oral.

Ce qu'on peut laisser tomber

NCurses, la state machine, les tests unitaires et les design patterns supplémentaires sont des bonus, sur 10 points maximum au total. Le `LevelParser` ne donne aucun point. ZombieSwimmer est à laisser au binôme.

Attention enfin au malus « code généré » (−10 par fonction) et à l'oral : écrire soi-même les fonctions, même en s'inspirant de ce plan, pour pouvoir les expliquer.